

fairyland_map

March 7, 2026

1 FAIRyland Spatial Visualization

This notebook creates a comprehensive spatial visualization of the FAIRyland archaeological dataset by combining RDF/Linked Data with geospatial analysis.

The visualization includes: - Archaeological features (color-coded by type) - Street network (highlighted infrastructure) - Special features (craters) - OpenStreetMap basemap for geographic context

Dataset: ../lod/fairyland.ttl

Author: Florian Thiery, Research Squirrel Engineers

License: CC-BY 4.0

Project: <https://github.com/Research-Squirrel-Engineers/FAIRyland>

1.1 Setup & Imports

```
[1]: # Import required libraries
import os
from rdflib import Graph, Namespace
import pandas as pd
import geopandas as gpd
from shapely import wkt
from shapely.geometry import Point
import matplotlib.pyplot as plt
import contextily as ctx
from matplotlib.patches import Patch
from matplotlib.lines import Line2D

# Jupyter-specific
%matplotlib inline
plt.rcParams['figure.dpi'] = 100

print(" Libraries imported")
```

Libraries imported

1.2 Configuration

Define RDF namespaces and file paths.

```
[2]: # RDF Namespaces
FAIRYLAND = Namespace("https://github.com/Research-Squirrel-Engineers/FAIRyland/
↳")
SUNI = Namespace("http://www.github.com/sparqlunicorn#")
GEO = Namespace("http://www.opengis.net/ont/geosparql#")

# File path - adjust if needed!
ttl_path = "../lod/fairyland.ttl"

print(" Configuration set")
```

Configuration set

1.3 Load RDF Dataset

```
[3]: print(f"Loading: {ttl_path}")

g = Graph()
g.parse(ttl_path, format="turtle")

print(f" Loaded {len(g):,} triples")
```

Loading: ../lod/fairyland.ttl

Loaded 1,847 triples

1.4 Query Spatial Features

Extract streets, craters, and archaeological features using SPARQL.

```
[4]: # Query 1: Street Network
street_query = """
PREFIX fairyland: <https://github.com/Research-Squirrel-Engineers/FAIRyland/>
PREFIX suni: <http://www.github.com/sparqlunicorn#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT DISTINCT ?feature ?name ?wkt WHERE {
    ?feature a fairyland:Street .
    OPTIONAL { ?feature suni:Name ?name }
    OPTIONAL {
        ?feature geo:hasGeometry ?geom .
        ?geom geo:asWKT ?wkt
    }
}
"""

streets = []
for row in g.query(street_query):
    if row.wkt:
```

```

        streets.append({
            "feature": str(row.feature),
            "name": str(row.name) if row.name else "Street",
            "wkt": str(row.wkt),
        })

print(f"Streets: {len(streets)}")

```

Streets: 7

```

[5]: # Query 2: Crater Features
crater_query = """
PREFIX suni: <http://www.github.com/sparqlunicorn#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT DISTINCT ?feature ?name ?wkt WHERE {
    ?feature suni:Name ?name .
    FILTER(REGEX(?name, "crater", "i"))
    OPTIONAL {
        ?feature geo:hasGeometry ?geom .
        ?geom geo:asWKT ?wkt
    }
}
"""

craters = []
for row in g.query(crater_query):
    if row.wkt:
        craters.append({
            "feature": str(row.feature),
            "name": str(row.name),
            "wkt": str(row.wkt)
        })

print(f"Craters: {len(craters)}")

```

Craters: 2

```

[6]: # Query 3: Archaeological Features
arch_query = """
PREFIX suni: <http://www.github.com/sparqlunicorn#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT DISTINCT ?feature ?name ?wkt WHERE {
    ?feature suni:Name ?name .
    OPTIONAL {
        ?feature geo:hasGeometry ?geom .
        ?geom geo:asWKT ?wkt
    }
}
"""

```

```

    }
    FILTER(?name != "Street")
}
"""

features = []
for row in g.query(arch_query):
    if row.wkt:
        wkt_str = str(row.wkt)
        if "POLYGON" in wkt_str.upper():
            try:
                geom = wkt.loads(wkt_str)
                centroid = geom.centroid
                features.append({
                    "feature": str(row.feature),
                    "type": str(row.name),
                    "lon": centroid.x,
                    "lat": centroid.y,
                })
            except:
                continue

print(f"Archaeological features: {len(features)}")

```

Archaeological features: 123

1.5 Create GeoDataFrames

Convert query results to GeoPandas DataFrames and reproject to Web Mercator (EPSG:3857).

```

[7]: # Archaeological features (point data)
df = pd.DataFrame(features)
geometry = [Point(xy) for xy in zip(df["lon"], df["lat"])]
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs="EPSG:4326")
gdf_mercator = gdf.to_crs("EPSG:3857")

print(f" Archaeological features: {len(gdf_mercator)} points")
display(gdf.head())

```

Archaeological features: 123 points

	feature	type	lon \
0	http://www.github.com/sparqlunicorn#0194d590-c...	Kötbullar	20.026396
1	http://www.github.com/sparqlunicorn#038d7342-a...	Kötbullar	20.026421
2	http://www.github.com/sparqlunicorn#03bf5218-b...	Kötbullar	20.026530
3	http://www.github.com/sparqlunicorn#069e7a2d-6...	Kötbullar	20.026371
4	http://www.github.com/sparqlunicorn#0800c7da-7...	Kötbullar	20.026380

lat geometry

```

0 63.880158 POINT (20.0264 63.88016)
1 63.880116 POINT (20.02642 63.88012)
2 63.880154 POINT (20.02653 63.88015)
3 63.880183 POINT (20.02637 63.88018)
4 63.880177 POINT (20.02638 63.88018)

```

```

[8]: # Street network (line data)
street_gdf_mercator = None
if streets:
    street_geometries = []
    for s in streets:
        try:
            geom = wkt.loads(s["wkt"])
            street_geometries.append(geom)
        except:
            pass

    if street_geometries:
        street_gdf = gpd.GeoDataFrame(
            streets[:len(street_geometries)],
            geometry=street_geometries,
            crs="EPSG:4326"
        )
        street_gdf_mercator = street_gdf.to_crs("EPSG:3857")
        print(f" Streets: {len(street_gdf_mercator)} segments")

# Crater features (point data)
crater_gdf_mercator = None
if craters:
    crater_geometries = []
    crater_data = []
    for c in craters:
        try:
            geom = wkt.loads(c["wkt"])
            centroid = geom.centroid
            crater_geometries.append(Point(centroid.x, centroid.y))
            crater_data.append(c)
        except:
            pass

    if crater_geometries:
        crater_gdf = gpd.GeoDataFrame(
            crater_data[:len(crater_geometries)],
            geometry=crater_geometries,
            crs="EPSG:4326"
        )
        crater_gdf_mercator = crater_gdf.to_crs("EPSG:3857")

```

```
print(f" Craters: {len(crater_gdf_mercator)} points")
```

Streets: 7 segments

Craters: 2 points

1.6 Create Spatial Visualization

Generate map with OpenStreetMap basemap and all feature layers.

```
[9]: # Create figure
fig, ax = plt.subplots(figsize=(18, 12))

# Set map extent
minx, miny, maxx, maxy = gdf_mercator.total_bounds
x_range = maxx - minx
y_range = maxy - miny
padding = 0.05
ax.set_xlim(minx - x_range * padding, maxx + x_range * padding)
ax.set_ylim(miny - y_range * padding, maxy + y_range * padding)

# Add OpenStreetMap basemap
ctx.add_basemap(
    ax,
    source=ctx.providers.OpenStreetMap.Mapnik,
    attribution_size=8,
    reset_extent=False,
    alpha=1.0,
)

# Layer 1: Streets
if street_gdf_mercator is not None and len(street_gdf_mercator) > 0:
    street_gdf_mercator.plot(ax=ax, color="white", linewidth=7, linestyle="--",
    ↪alpha=0.6, zorder=4)
    street_gdf_mercator.plot(ax=ax, color="#FF1493", linewidth=5,
    ↪linestyle="--", alpha=0.95, zorder=5)

# Layer 2: Craters
if crater_gdf_mercator is not None and len(crater_gdf_mercator) > 0:
    crater_gdf_mercator.plot(ax=ax, color="orange", markersize=400, alpha=0.3,
    ↪edgecolor="none", zorder=14)
    crater_gdf_mercator.plot(ax=ax, color="#FF8C00", markersize=350, alpha=0.
    ↪85, edgecolor="#FF4500", linewidth=3, zorder=15)

# Layer 3: Archaeological features (color-coded)
unique_types = gdf["type"].unique()
colors = plt.cm.Set3.colors[:len(unique_types)]
type_colors = {ftype: colors[idx] for idx, ftype in enumerate(unique_types)}
```

```

patches = []
for ftype, color in type_colors.items():
    type_data = gdf_mercator[gdf_mercator["type"] == ftype]
    if ftype.lower() == "crater":
        continue

    # Drop shadow
    type_data.plot(ax=ax, color="black", markersize=145, alpha=0.35,
↳edgecolor="none", zorder=9)
    # Main marker
    type_data.plot(ax=ax, color=color, markersize=140, alpha=0.9,
↳edgecolor="black", linewidth=1.8, zorder=10)

    patches.append(Patch(facecolor=color, label=f"{ftype}
↳(n={len(type_data)})", edgecolor="black", linewidth=0.5))

# Build legend
legend_elements = patches + [
    Line2D([0], [0], color="#FF1493", linewidth=4, linestyle="--",
↳label=f"Street (n={len(streets)})", alpha=0.95),
    Line2D([0], [0], marker="o", color="w", label=f"Crater (n={len(craters)})",
        markerfacecolor="#FF8C00", markeredgecolor="#FF4500", markersize=13,
↳alpha=0.85, markeredgewidth=2)
]

# Styling
ax.set_axis_off()

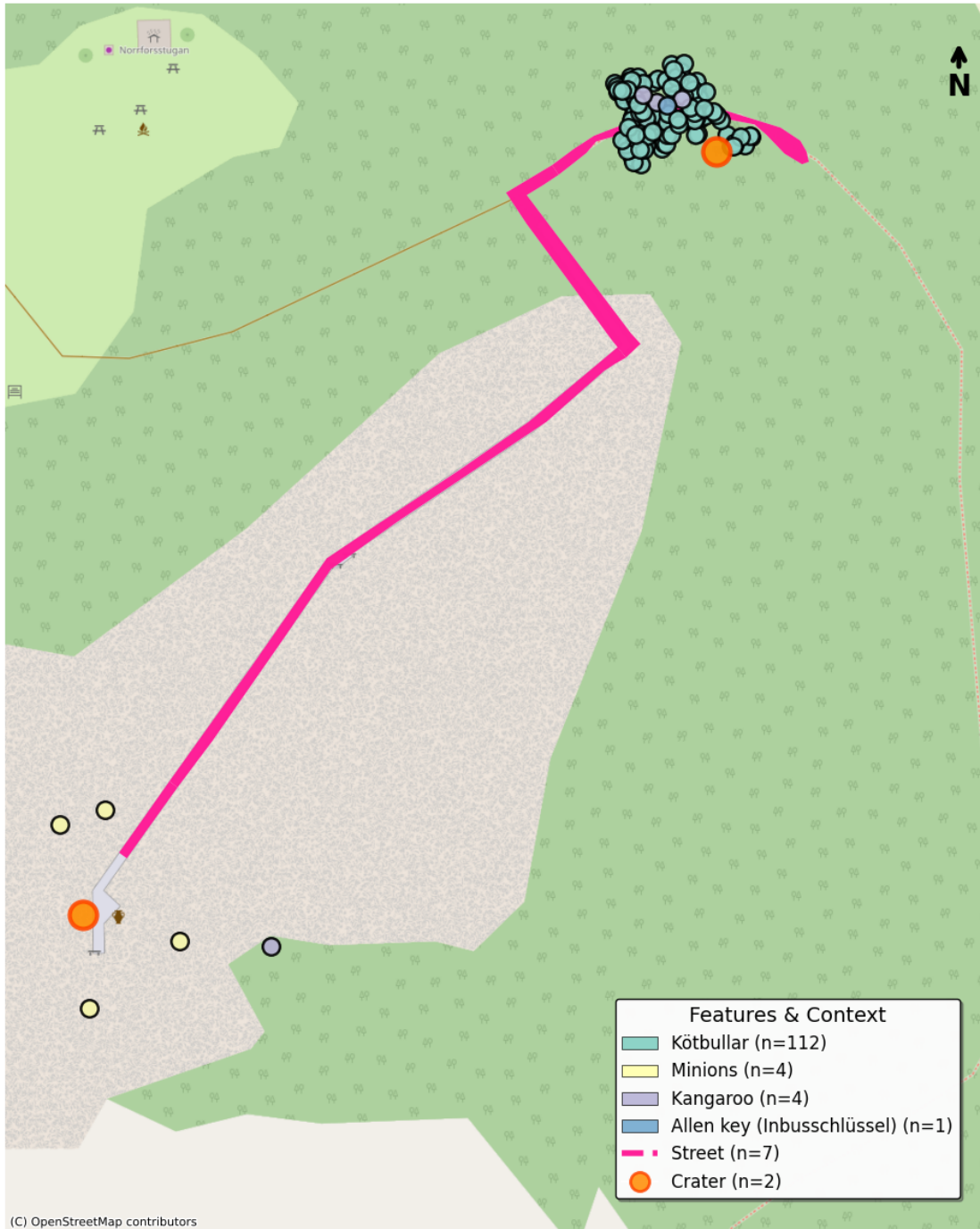
# North arrow
ax.annotate("N", xy=(0.97, 0.97), xytext=(0.97, 0.92), xycoords="axes fraction",
        fontsize=20, fontweight="bold", ha="center", va="bottom",
        arrowprops=dict(arrowstyle="->", lw=3.5, color="black"))

# Legend
plt.legend(handles=legend_elements, title="Features & Context",
↳title_fontsize=14, fontsize=12,
        loc="lower right", framealpha=0.97, edgecolor="black", fancybox=True,
↳shadow=True,
        bbox_to_anchor=(0.98, 0.02))

plt.tight_layout()
plt.show()

print(" Map created successfully")

```



Map created successfully

1.7 Summary Statistics

```
[10]: print("Dataset Statistics:")
      print(f" → Archaeological features: {len(gdf)} (color-coded by type)")
      print(f" → Street segments: {len(streets)}")
      print(f" → Crater features: {len(craters)}")
      print("\nVisualization Details:")
      print(f" → CRS: EPSG:4326 → EPSG:3857")
      print(f" → Basemap: OpenStreetMap")
```

Dataset Statistics:

- Archaeological features: 123 (color-coded by type)
- Street segments: 7
- Crater features: 2

Visualization Details:

- CRS: EPSG:4326 → EPSG:3857
- Basemap: OpenStreetMap